

DAY 1 · 월요일

LLM 의 원리와 이해

AI 에이전트의 핵심인 LLM 의 작동 원리 · 프롬프트 · 한계를 이해한다

담당 세션 (1)

오후 특강 2

14:30-16:00 · 90 분

LLM 의 원리와 이해

전체 과정 로드맵

DAY 1

월요일

LLM 원리와 이해

● 진행 중

DAY 2

화요일

에이전트 원리 + 클로드
코드

DAY 3

수요일

미니 프로젝트 + 리서치
이론 · 팀

DAY 4

목요일

팀 리서치 — 기획 · 구현 ·
실행

DAY 5

금요일

실행 · 분석 + 보고서 ·
발표

진행 단계 : 이론 → 도구 → 실전 → 결과 → 발표

오후 특강 2 14:30-16:00 · 90 분

LLM 의 원리와 이해

LLM 이 무엇을 어떻게 하는지 — 토큰 예측 · 학습 · 프롬프트 · 한계를 이해한다 .

LLM 의 원리와 이해

이 세션에서 다룰 내용

1 언어 모델 간단 역사

2 LLM 이란 + 토큰 · 예측

3 학습 : 사전학습 · 정렬

4 추론 (사고) 모델

5 임베딩 & 의미 검색

6 RAG — 검색 증강 생성

7 프롬프트 엔지니어링

8 생성 제어 · 컨텍스트 윈도우

9 능력과 한계 · 환각

10 모델 선택

11 LLM → 에이전트

이 세션에서 다룰 것

- LLM 의 작동 원리

토큰 예측 · 학습 · 추론

- 외부 지식과 결합

임베딩 · 의미 검색 · RAG → 리서치 에이전트의 토대

- 제어와 한계

프롬프트 · 모델 선택 · 환각 등 주의점

언어 모델 간단 역사



NOTE 초기 통계 (n-gram) → 신경망 → Transformer 로 도약 — 이제 각 단계를 하나씩

word2vec (2013) — 단어를 벡터로

- **무엇**

단어를 의미가 담긴 숫자 벡터로 표현 (Google)

- **왜 중요**

의미 연산이 가능 — 'king - man + woman $\approx$ queen'

- **한계**

단어 단위 · 문맥 (앞뒤 의미) 을 잘 못 담음

NOTE 오늘 배울 '임베딩'의 출발점

Transformer (2017) — Attention

- **무엇**

논문 'Attention is All You Need' 의 새 신경망 구조

- **핵심**

문장 속 단어 간 관계를 한 번에 · 병렬로 파악 (attention)

- **의의**

이전 방식 (RNN) 보다 빠르고 긴 문맥 처리 → 현대 LLM 의 토대

NOTE 지금의 GPT·Claude 모두 이 구조 위에

GPT·BERT (2018~) — 사전학습 시대

- GPT

생성형 — 다음 토큰 예측 (말 잇기)

- BERT

이해형 — 빈칸 채우기 (문맥 파악)

- 패러다임

대규모 사전학습 → 작업별 미세조정 · 모델 · 데이터 키우기 시작

NOTE '사전학습'이라는 오늘의 핵심 개념이 자리잡음

GPT-3 → ChatGPT (2020·2022) — 규모와 정렬

- GPT-3 (2020)

175B 파라미터 · few-shot — '규모의 힘' 입증

- ChatGPT (2022.11)

정렬 (RLHF) + 대화 인터페이스

- 의의

전문가 도구 → 누구나 쓰는 AI 로 폭발적 대중화

NOTE 능력 (사전학습) 과 말귀 (정렬) 가 합쳐진 순간

추론 · 멀티모달 (2024~) — 최신 흐름

- **추론 모델**

바로 답하지 않고 단계적으로 '생각' (o1·DeepSeek-R1)

- **멀티모달**

텍스트 + 이미지 · 음성까지 함께 처리

- **에이전트로**

도구 · 반복을 붙여 '행동하는 AI' → Day 2 주제

NOTE 오늘 우리가 쓰는 모델들이 여기

LLM이란 무엇인가

- **Large Language Model**

방대한 텍스트로 학습한 대규모 신경망 — 언어를 생성 · 처리

- **핵심 동작**

지금까지의 텍스트로 '다음에 올 토큰'을 확률로 예측

- **'이해'가 아니라 '패턴'**

의미를 아는 게 아니라 통계적 패턴을 재현하는 것

NOTE 예 : '커피를 마시면 잠이 ____' 다음에 올 토큰을 확률로 고른다

입력에서 출력까지



NOTE 한 번에 한 토큰씩, 반복해서 생성 (autoregressive) — 이제 각 단계를 하나씩

토큰화 — 텍스트를 토큰으로

- 토큰이란

단어 또는 단어 조각 (글자보다 크고 단어보다 작을 수 있음)

- 왜 토큰

어휘를 효율적으로 다루기 위해 — 모든 단어를 외우지 않아도 됨

- 길이 · 비용

영어 ~4 글자 $\approx$ 1 토큰 · 한글은 더 잘게 $\rightarrow$ 토큰 수가 비용 · 길이를 결정

NOTE 예 : 'AI 에이전트' $\rightarrow$ 여러 토큰으로 쪼개짐

예측 — 다음 토큰의 확률

- 무엇을 계산

지금까지의 토큰을 보고 '다음에 올 토큰'의 확률 분포

- 예

'커피를 마시면 잠이 ____' → 안 (0.6) · 온다 (0.2) · ...

- 핵심

여기엔 '정답'이 아니라 '확률'만 있다

NOTE 모델이 아는 건 사실이 아니라 '그럴듯함' — 환각의 뿌리

샘플링 — 토큰 하나 선택

- 선택 방식

확률 분포에서 토큰 1 개를 고름

- 결정적 vs 다양

가장 높은 것만 ? 또는 확률대로 무작위 ?

- temperature

이 다양성을 조절하는 손잡이 → 뒤 ' 생성 제어 ' 에서

NOTE 같은 질문에 매번 다른 답이 나오는 이유

반복 — 끝까지 생성

- **이어가기**

고른 토큰을 문장에 붙이고 → 다시 예측 (1~3 반복)

- **autoregressive**

한 번에 한 토큰씩, 끝 신호까지

- **함의**

긴 답일수록 앞에서의 작은 흔들림이 누적될 수 있음

NOTE 그래서 긴 출력은 중간 점검·검증이 중요

사전학습 (Pre-training)

- **방대한 텍스트로 자기지도 학습**

웹 · 책 · 코드에서 ' 다음 토큰 맞추기 ' 를 반복

- **무엇을 배우나**

문법 · 세상 지식 · 추론 패턴 · 문체까지 광범위하게

- **한계의 씨앗**

학습 시점 이후 정보는 모름 · 데이터 편향을 흡수

NOTE 비유 : 거대한 빈칸 채우기를 수십억 번 — 그 과정에서 지식이 ' 딸려 ' 학습된다

정렬 (Alignment)

- 사전학습만으론 부족

'유용하고 · 안전하게 · 지시를 따르도록' 추가로 다듬음

- instruction tuning

지시-응답 예시로 '시키는 대로' 하도록 미세조정

- 인간 피드백 (RLHF 등)

사람이 선호하는 답을 강화 — 도움되고 해롭지 않게

NOTE 사례 : InstructGPT(2022) — 정렬한 1.3B 답을 175B GPT-3 보다 사람이 선호 (~71%)

추론 (사고) 하는 모델

- 일반 생성 vs 추론

바로 답하지 않고 , 단계적으로 ' 생각 ' 한 뒤 답한다

- 언제 강한가

복잡한 문제 · 다단계 추론 · 코딩

- 트레이드오프

더 정확하지만 더 느리고 비싸다

NOTE 사례 : DeepSeek-R1(2025· 오픈)·OpenAI o1→o3 — AIME 78%→96.7% (벤치마크는 빠르게 바뀜)

임베딩 & 의미 검색

- 임베딩이란

텍스트를 의미를 담은 숫자 벡터로 변환

- 의미 검색

글자가 달라도 '뜻이 비슷한' 문서를 찾음 (키워드 매칭의 한계 극복)

- 어디에 쓰나

방대한 문헌에서 관련 자료 검색 — 리서치 에이전트의 토대

NOTE 키워드가 아니라 '의미'로 찾는다 · 사례 : Etsy·Spotify 검색 / 추천이 임베딩 기반

검색 증강 생성 (RAG)



NOTE 사례 : OpenEvidence — 동료심사 문헌 (NEJM·JAMA) 기반 의사용 Q&A (약학 RAG 예)

프롬프트가 동작을 바꾼다

- **프롬프트 = 모델에 주는 입력 전체**

지시 + 맥락 + 예시 + 데이터

- **좋은 프롬프트**

명확한 목표 · 충분한 맥락 · 원하는 형식 명시

- **나쁜 프롬프트**

모호 · 맥락 부족 → 엉뚱하거나 일반적인 답

NOTE 비교 : ' 코딩 도와줘 ' vs 'CSV 항목별 합계 코드 만들고 실행 · 확인해줘 '

자주 쓰는 패턴

- 역할 · 맥락 부여

'너는 ~ 전문가' + 배경 → 답의 톤 · 깊이 조정

- 예시 제공 (few-shot)

원하는 입출력 예 2~3 개 → 형식이 안정

- 단계적 사고 (CoT)

'단계별로 생각해줘' → 복잡한 추론 정확도↑

NOTE 사례 : GPT-3(2020) — 예시 몇 개 (few-shot) 만으로 번역 · Q&A 수행

같은 모델, 다른 출력

- temperature

낮으면 결정적 · 일관, 높으면 다양 · 창의 (불안정)

- 확률적 생성

같은 질문도 매번 다른 답이 나올 수 있다

- 재현성에의 함의

리서치엔 낮은 temperature 가 유리 — 결과 안정

컨텍스트 윈도우

- 한 번에 볼 수 있는 토큰 한계

프롬프트 + 대화 + 자료가 이 한도 안에 들어가야

- 길어지면

비용↑ · 속도↓ · 중간 내용 놓침 (lost in the middle)

- 에이전트와의 관련

외부 메모리 · 검색 · 도구로 한계를 보완

NOTE 현재 Claude 최대 1M 토큰 (≈ 책 여러 권) — 긴 문서 · 코드 통째 분석 (단 lost in the middle 주의)

LLM 의 능력과 한계

잘하는 것

- 요약 · 번역 · 분류
- 코드 생성 · 설명
- 텍스트 추출 · 재구성
- 브레인스토밍

주의할 것

- 환각 — 없는 사실 지어냄
- 최신 · 실시간 정보
- 정확한 계산 · 셈
- 일관된 장기 추론

환각 (hallucination) 바로 알기

- 그럴듯하지만 틀린 출력

출처 · 인용 · 수치를 자신있게 지어내기도

- 왜 생기나

'그럴듯한 다음 토큰'을 생성할 뿐 사실 검증을 안 함

- 대응

출처 확인 · 검색 연동 (RAG) · 교차검증 → 리서치 에이전트의 존재 이유

NOTE 실제 사례 : 변호사가 ChatGPT 가짜 판례 인용→벌금 (Mata v. Avianca, 2023) · 에어캐나다 챗봇이 없는 환불정책 지어내 배상 (2024)

어떤 모델을 쓸까

- 크기 ↔ 성능

큰 모델 = 똑똑하지만 느리고 비쌈

- 작업에 맞게

간단한 작업엔 작고 빠른 모델, 어려운 작업엔 크고 / 추론 모델

- Claude 예

Opus(최상위) · Sonnet(균형) · Haiku(빠름 · 저비용)

NOTE 참고 가격 (~2026.6, 1M 토큰 입력 / 출력): Opus \$5/\$25 · Sonnet \$3/\$15 · Haiku \$1/\$5 (변동)

LLM → 에이전트로 가는 길

- LLM 단독의 한계

도구 없음 · 기억 없음 · 한 번의 응답으로 끝

- RAG·검색을 루프로 = 에이전트

찾고 → 검증하고 → 종합 — 리서치 에이전트로

- 다음 : Day 2

에이전트의 정의와 Claude Code 실습

NOTE 오늘의 임베딩 · RAG 가 리서치 에이전트의 토대다

핵심 정리 및 다음 과정

핵심 정리

- LLM 의 핵심은 ' 다음 토큰 예측 ' — 확률적 생성이다
- 임베딩 · RAG 로 외부 지식과 결합하면 환각↓ · 최신성↑ (리서치 에이전트의 토대)
- LLM 은 한계가 있다 → 도구 · 검색 · 반복 (에이전트) 이 필요하다

다음 과정 — Day 2

AI 에이전트의 정의 (LLM+tools+loop) 와
리서치 에이전트 개요를 배우고 , Python 과
Claude Code 를 설치 · 설정합니다 .